Spécifications techniques

Menu Maker by Qwenta

Version	Auteur	Date	Approbation
1.1	Julien Holtzinger, Webgencia	30/09/2026	En attente de validation - Soufiane, Webgencia

Le but de ce document est de définir les spécifications techniques de l'outil “Menu Maker by Qwenta”, à partir des spécifications fonctionnelles et de la maquette fournies par le client, ainsi que de la veille technologique menée en parallèle.

I. Choix technologiques — page 1

II. Liens avec le back-end — page 4

III. Préconisations concernant le domaine et l'hébergement — page 4

IV. Accessibilité — page 5

V. Recommandations en termes de sécurité — page 5

VI. Maintenance du site et futures mises à jour — page 5

I. Choix technologiques

État des lieux des besoins fonctionnels et de leurs solutions techniques :

Besoin	Contraintes	Solution	Description de la solution	Justification
Interface de l'application	L'outil doit être dynamique (spécifications fonctionnelles), en version desktop, et compatible avec les dernières versions de Chrome, Safari et Firefox.	**React (avec Vite)**	L'interface est découpée en composants réutilisables (Sidebar, Header, Footer, modales, formulaires...). L'état de React met à jour uniquement les parties de la page qui changent, sans rechargement. Vite sert d'outil de développement et de construction du projet.	1) Les composants réutilisables sont développés une seule fois et utilisés dans toute l'application, ce qui fait gagner du temps et facilite la maintenance. 2) La gestion de l'état par React permet une interface dynamique, sans rechargement de page, ce qui répond directement à la demande du client.
Landing non connectée	Page publique, accessible sans compte, qui présente l'outil à l'internaute à travers trois sections : la bannière, « Personnalisez votre menu » et les explications étape par étape (« Comment créer votre menu ? »).	**Composants React (Header, Bannière, sections, Footer) + React Router.**	La Landing est composée de composants React réutilisables : le Header, la Bannière, la section « Personnalisez votre menu », la section « Comment créer votre menu ? » et le Footer. Leur contenu, identique pour tous les visiteurs, est statique et intégré directement au front-end, sans appel à l'API. React Router définit les routes publiques (accueil, connexion), séparées des routes protégées de l'espace connecté.	1) Le découpage en composants React rend chaque section indépendante, ce qui facilite sa modification et sa réutilisation. 2) Le contenu étant statique, aucune requête API n'est nécessaire, ce qui accélère le chargement de la page d'accueil, la première impression des futurs clients.
Page login	Dans une modale, l'utilisateur saisit uniquement son adresse e-mail. Un e-mail lui est envoyé pour s'authentifier s'il a déjà un compte, ou pour confirmer son adresse et créer automatiquement son compte s'il est nouveau. Un lien « Besoin d'aide » permet d'écrire directement à l'équipe.	**React-modal + authentification sans mot de passe par lien magique : Nodemailer (envoi du lien) + JWT (Node.js / Express) + MongoDB.**	La modale de connexion (React-modal) contient un champ e-mail. À la validation, le back-end génère un lien unique et temporaire, envoyé via Nodemailer. Au clic sur ce lien, le back-end le vérifie, crée le compte dans MongoDB si l'adresse est nouvelle, puis génère un token JWT qui maintient la session. Le lien « Besoin d'aide » ouvre directement la messagerie de l'utilisateur vers l'adresse de l'équipe.	1) Aucun mot de passe n'est stocké en base de données, ce qui supprime le risque de fuite de mots de passe ; seul le propriétaire de la boîte mail peut recevoir le lien de connexion. 2) Un parcours unique sert à la fois à la connexion et à l'inscription, sans mot de passe à retenir, ce qui simplifie l'accès et fait gagner du temps au restaurateur.

Besoin	Contraintes	Solution	Description de la solution	Justification
Catégorie de plat	Depuis l'écran de création de menu, le restaurateur doit pouvoir créer une nouvelle catégorie dans une modale dédiée, la valider, puis sélectionner une catégorie créée précédemment.	**React-modal + React Hook Form ; route API protégée par le middleware JWT (Node.js / Express) ; catégories stockées dans MongoDB.**	Au clic sur « + Catégorie », React-modal ouvre la modale « Ajouter une catégorie », dont le champ est géré par React Hook Form. À la validation, la catégorie est envoyée au back-end, qui l'enregistre dans MongoDB, associée au restaurateur. Elle apparaît immédiatement dans la liste et dans l'aperçu du menu, sans rechargement de la page. Les catégories déjà créées restent disponibles à la sélection pour les menus suivants.	1) Les catégories sont enregistrées en base de données, ce qui permet au restaurateur de les réutiliser d'un menu à l'autre sans les ressaisir. 2) React-modal et React Hook Form sont déjà utilisés dans le projet, ce qui assure une expérience homogène et limite le nombre de dépendances.
Création de plat	Après avoir sélectionné une catégorie, qu'il peut modifier, le restaurateur saisit ses plats dans une modale défilante. Chaque plat peut avoir une photo (2 Mo maximum), un nom, un prix et une description. Le nombre de plats par catégorie est illimité.	**React-modal + React Hook Form (liste de champs dynamique) ; Multer pour l'upload des photos (images uniquement, 2 Mo maximum) ; route API protégée par le middleware JWT (Node.js / Express) ; plats et chemins des photos stockés dans MongoDB.**	Au clic sur « + Plats » sous une catégorie, React-modal ouvre une modale défilante. React Hook Form gère une liste dynamique de plats : le restaurateur peut en ajouter autant qu'il le souhaite, chacun avec photo, nom, prix et description. À la validation, les photos sont reçues par Multer et enregistrées sur le serveur ; les plats et les chemins des photos sont sauvegardés dans MongoDB. Les plats apparaissent immédiatement dans la liste et dans l'aperçu du menu.	1) React Hook Form gère nativement les listes de champs dynamiques et la validation (prix numérique, champs obligatoires), ce qui évite d'ajouter une librairie supplémentaire. 2) Multer limite la taille et le type des fichiers reçus, ce qui protège le serveur contre les fichiers trop lourds ou malveillants.
Style de menu	Le restaurateur doit pouvoir visualiser le menu en cours de création, choisir une typographie parmi trois propositions et une couleur de texte. Les changements doivent être visibles immédiatement dans l'aperçu.	**Variables CSS (custom properties) pilotées par l'état React + sélecteur de couleur natif HTML ; choix enregistrés avec le menu dans MongoDB.**	Le restaurateur sélectionne l'une des trois typographies proposées et une couleur de texte, par défaut celle définie dans son branding. Ces choix sont gardés dans l'état React, qui met à jour deux variables CSS appliquées à l'aperçu : tout le texte du menu change instantanément, sans rechargement. Les choix sont enregistrés avec le menu dans MongoDB.	1) Les variables CSS sont gérées nativement par tous les navigateurs, ce qui permet un changement de style instantané sans ajouter de librairie. 2) Le style du menu ne dépend d'aucune librairie externe, ce qui allège l'application et évite d'avoir des mises à jour supplémentaires à surveiller.
Exportation PDF	Le restaurateur doit pouvoir télécharger en un clic le fichier PDF de son menu, depuis l'encart « Exporter en .pdf ». Le PDF doit conserver la mise en forme choisie (typographie, couleurs, logo).	**@react-pdf/ renderer (génération du PDF côté front-end).**	Le menu est décrit avec les composants de @react-pdf/renderer, qui reprennent ses données (catégories, plats, prix), le logo du restaurant ainsi que la typographie et la couleur choisies. Au clic sur « Exporter en .pdf », le fichier est généré directement dans le navigateur, puis téléchargé.	1) @react-pdf/renderer génère un vrai PDF, avec du texte et non une image, ce qui garantit une qualité d'impression optimale, en cohérence avec le métier d'imprimeur de Qwenta. 2) La génération se fait dans le navigateur, ce qui évite de solliciter le serveur pour cette fonctionnalité.
Commander l'impression d'un menu	Depuis le Dashboard, l'encart « Imprimer un menu » doit ouvrir, en un clic et dans un nouvel onglet, le back-office d'impression de Qwenta.	**Lien externe sécurisé (noopener) ; URL du back-office centralisée dans une constante de configuration.**	Sur le Dashboard, l'encart « Imprimer un menu » est un lien externe qui ouvre le back-office de Qwenta dans un nouvel onglet, avec l'attribut noopener. La commande est ensuite entièrement gérée par Qwenta. L'URL, à confirmer avec Qwenta, est centralisée dans une constante de MenuMaker.	1) La commande est gérée directement dans le back-office existant de Qwenta, ce qui évite de redévelopper un système de commande et s'appuie sur l'outil qu'ils maîtrisent déjà. 2) L'URL est centralisée dans une constante, ce qui permet de la modifier en un seul endroit.
Menus précédents	Au clic sur « Mes menus », le restaurateur accède à la liste de ses menus créés précédemment, avec leur date de création. Il peut modifier ou supprimer chaque menu, et créer un nouveau menu depuis la même vue.	**Routes API CRUD (GET / POST / PUT / DELETE) protégées par le middleware JWT (Node.js / Express) ; menus stockés dans MongoDB avec dates de création automatiques (option timestamps de Mongoose) ; React Router pour la navigation vers l'éditeur.**	La page « Mes menus » récupère via l'API la liste des menus du restaurateur connecté et affiche pour chacun une miniature et sa date de création. « Modifier » ouvre le menu dans l'éditeur grâce à React Router ; « Supprimer » efface le menu après confirmation ; « Ajouter un menu » lance la création d'un nouveau menu.	1) La structure imbriquée d'un menu (catégories, plats) se prête naturellement au format document de MongoDB, ce qui simplifie son enregistrement et sa lecture. 2) L'option timestamps de Mongoose ajoute automatiquement la date de création, ce qui évite de la gérer manuellement.

Besoin	Contraintes	Solution	Description de la solution	Justification
Informations légales	Le lien doit être accessible depuis les pages **connectées et déconnectées**, et « Tous droits réservés » doit figurer **sur toutes les pages**.	**Composants Footer et Sidebar + React-modal**	Sur les pages publiques, le composant Footer affiche le lien « Mentions légales » et la mention « Tous droits réservés » ; dans l'espace connecté, ces deux éléments figurent en bas du composant Sidebar. Au clic sur le lien, React-modal ouvre une modale contenant le texte des mentions légales. Ce contenu étant statique, il est directement intégré au composant, sans appel à l'API.	1) Le Footer et la Sidebar ne sont développés qu'une seule fois, ce qui affiche automatiquement le lien et la mention sur toutes les pages, connectées et déconnectées. 2) Le contenu étant statique, aucune requête API n'est nécessaire, ce qui accélère l'ouverture de la modale et garantit son affichage même en cas de panne du back-end.
Tarifs	L'internaute doit pouvoir cliquer sur l'onglet « Tarifs », qui ouvre la page tarifs du site de Qwenta dans un nouvel onglet. L'URL définitive n'est pas encore connue.	**Lien externe dans le Header + ouverture sécurisée dans un nouvel onglet (noopener).**	Le composant Header affiche, pour les internautes non connectés, un onglet « Tarifs ». Au clic, un lien externe ouvre la page tarifs de Qwenta dans un nouvel onglet, avec l'attribut noopener pour sécuriser l'ouverture. L'URL, pas encore définie, est centralisée dans une constante de configuration de MenuMaker, afin d'être mise à jour en un seul endroit dès que Qwenta la communiquera.	1) La page Tarifs est gérée directement par Qwenta, ce qui garantit des prix toujours à jour, sans intervention de notre part. 2) L'URL est centralisée dans une constante, ce qui permet de la modifier en un seul endroit dès que l'URL définitive sera connue.
Exportation Deliveroo	Le restaurateur doit pouvoir exporter son menu vers Deliveroo en un clic, via l'encart « Diffuser sur Deliveroo » de la catégorie « Exportez et diffusez », puis être redirigé vers l'application Deliveroo. Le mode de connexion avec Deliveroo reste à définir.	**Menu API de Deliveroo, appelée par le back-end (Node.js / Express), avec les clés d'accès stockées dans des variables d'environnement.**	Au clic sur l'encart « Diffuser sur Deliveroo », le front-end de MenuMaker envoie une requête au back-end. Celui-ci récupère le menu en base de données et l'envoie à la Menu API de Deliveroo grâce aux clés d'accès sécurisées. Une fois l'envoi confirmé, le restaurateur est redirigé vers l'application Deliveroo.	1) L'appel à l'API Deliveroo passe par le back-end, ce qui garantit que les clés d'accès ne sont jamais exposées dans le navigateur. 2) Le menu est transmis automatiquement à Deliveroo en un clic, ce qui évite au restaurateur de le recopier à la main et supprime les risques d'erreur ou d'oubli lors des mises à jour.
Partage Instagram	Le restaurateur doit pouvoir partager son menu sur Instagram en un clic, via l'encart « Partager sur Instagram » de la catégorie « Exportez et diffusez ». Au clic, des images du menu au format carré sont générées, puis le restaurateur, après avoir autorisé l'accès à son compte Instagram, y est redirigé avec ses photos. Le mode de connexion avec Instagram reste à définir.	**html2canvas (génération des images carrées côté front-end) + API Instagram, appelée par le back-end (Node.js / Express) après autorisation du restaurateur via OAuth 2.0 ; clés d'accès stockées dans des variables d'environnement.**	Au clic sur l'encart « Partager sur Instagram », html2canvas convertit l'aperçu du menu en images au format carré (1080 × 1080 px) côté front-end. Si le restaurateur n'a pas encore connecté son compte, une fenêtre d'autorisation OAuth 2.0 lui permet d'autoriser MenuMaker à publier en son nom. Les images sont ensuite envoyées au back-end, qui les héberge puis les transmet à l'API Instagram. Enfin, le restaurateur est redirigé vers son compte Instagram.	1) L'autorisation OAuth 2.0 donne à MenuMaker un accès limité à la seule publication, sans jamais connaître le mot de passe du restaurateur, ce qui respecte le principe du moindre privilège et renforce la sécurité de son compte. 2) html2canvas convertit directement l'aperçu déjà personnalisé, ce qui évite de redévelopper la mise en page et garantit une image identique au menu affiché.
Déconnexion	Le bouton de déconnexion doit être accessible **sur toutes les pages** de l'espace connecté.	**Composant Sidebar + JWT + React Router**	Un composant Sidebar, commun à toutes les pages connectées, affiche le bouton de déconnexion. Au clic, le token JWT stocké dans le navigateur est supprimé, puis React Router redirige l'utilisateur vers la page de connexion.	1) Le bouton n'est développé qu'une seule fois, dans le composant Sidebar, et s'affiche automatiquement sur toutes les pages connectées. 2) Toute modification du bouton se fait à un seul endroit et s'applique à toutes les pages, ce qui facilite la maintenance et évite les oublis ou les incohérences.

Besoin	Contraintes	Solution	Description de la solution	Justification
Infos utilisateur	Le restaurateur doit pouvoir modifier ses informations, lier plusieurs adresses e-mail à son compte et modifier son adresse e-mail principale. Le changement d'adresse principale doit être sécurisé, pour empêcher qu'un tiers s'approprie le compte.	**React Hook Form (formulaire) + route API protégée par le middleware JWT (Node.js / Express) + champ tableau d'emails dans le schéma Mongoose + Nodemailer (email de confirmation lors du changement d'adresse principale).**	Depuis l'onglet « Mon compte » de la Sidebar, le restaurateur modifie ses informations via un formulaire géré par React Hook Form. Pour changer l'adresse principale, le back-end vérifie le token JWT et que la nouvelle adresse n'est pas déjà utilisée, puis envoie via Nodemailer un lien de confirmation unique et temporaire à la nouvelle adresse, ainsi qu'une demande de validation à l'ancienne. L'adresse n'est mise à jour dans MongoDB qu'après ces deux confirmations. Les adresses secondaires sont stockées dans un champ tableau du schéma utilisateur.	1) La double confirmation (nouvelle et ancienne adresse) garantit que l'adresse saisie est valide et que seul le propriétaire du compte peut la modifier, ce qui empêche un tiers de s'approprier le compte. 2) React Hook Form et MongoDB sont déjà utilisés dans le projet, ce qui fait gagner du temps à l'équipe, qui maîtrise ces outils, et limite le nombre de dépendances à maintenir.
Dashboard	Le Dashboard doit afficher le logo et le nom du restaurant connecté, les encarts « Créer un menu », « Diffuser un menu » et « Imprimer un menu », ainsi qu'une section « Pour aller plus loin » présentant les 3 derniers articles du blog de Qwenta consacrés à MenuMaker (titre, photo de couverture et lien).	**Route API protégée par le middleware JWT (Node.js / Express) et MongoDB pour le logo et le nom du restaurant ; React Router pour les encarts ; lecture du flux RSS du blog de Qwenta par le back-end avec rss-parser, avec mise en cache.**	Le Dashboard affiche le logo et le nom du restaurant, récupérés en base de données via une route API protégée. Les trois encarts, identiques pour tous les restaurateurs, sont intégrés directement au front-end et redirigent vers les pages correspondantes grâce à React Router. La section « Pour aller plus loin » affiche les 3 derniers articles consacrés à MenuMaker, que le back-end récupère depuis le flux RSS du blog de Qwenta et garde en cache. Le flux RSS du blog reste à confirmer avec Qwenta.	1) La lecture du flux RSS met automatiquement à jour la section « Pour aller plus loin » à chaque nouvelle publication de Qwenta, ce qui évite toute saisie manuelle des articles. 2) La mise en cache des articles réduit les requêtes inutiles vers le blog de Qwenta, ce qui accélère l'affichage du Dashboard et garantit son affichage même en cas de panne du blog.
Branding restaurateur	Le restaurateur doit pouvoir ajouter, modifier et supprimer le logo de son restaurant ainsi que ses couleurs de base.	**Formulaire React Hook Form avec sélecteur de couleur natif HTML ; routes API CRUD (POST / PUT / DELETE) protégées par le middleware JWT (Node.js / Express) ; Multer pour l'upload du logo ; chemin du logo et codes couleur stockés dans MongoDB.**	Depuis l'onglet « Mon restaurant » de la Sidebar, le restaurateur ajoute, modifie ou supprime son logo et ses couleurs de base via un formulaire React Hook Form. Le logo est reçu par Multer, qui l'enregistre sur le serveur ; son chemin et les codes couleur sont sauvegardés dans MongoDB, via des routes protégées par le middleware JWT. Ces éléments sont ensuite réutilisés sur le Dashboard et dans l'aperçu des menus.	1) Seul le chemin du logo est stocké dans MongoDB, l'image restant sur le serveur, ce qui évite de surcharger la base de données et préserve ses performances. 2) Le logo et les couleurs sont définis une seule fois dans « Mon restaurant », ce qui évite au restaurateur de les ressaisir à chaque menu et garantit une identité visuelle cohérente sur tous ses supports.

II. Liens avec le back-end

- Langage pour le serveur : Node.js, avec le framework Express.
- Besoin d'une API : oui, une API REST développée avec Express et consommée par le front-end React.
- Base de données choisie : MongoDB (NoSQL), via l'ODM Mongoose.

Justification : la structure imbriquée des menus (catégories > plats) se prête naturellement au format document de MongoDB, et l'ensemble de la stack (React, Node.js, MongoDB) repose sur un seul langage, JavaScript, ce qui simplifie le développement et la maintenance.

III. Préconisations concernant le domaine et l'hébergement

- Nom de domaine : sous-domaine de qwenta.com (ex. menumaker.qwenta.com), en cours de validation avec le client (cf. spécifications fonctionnelles).
- Nom de l'hébergement : un hébergeur compatible Node.js et MongoDB (ex. Render ou Railway pour le serveur, MongoDB Atlas pour la base de données) — à confirmer selon le budget défini avec Qwenta.
- Adresses e-mail : à créer sur le nom de domaine retenu (ex. contact@menumaker.qwenta.com) pour les communications officielles liées à l'outil.

IV. Accessibilité

- Compatibilité navigateur : dernières versions de Chrome, Safari et Firefox (cf. spécifications fonctionnelles).
- Types d'appareils : version desktop uniquement ; aucune version mobile n'est prévue à ce stade.
- Niveau d'accessibilité minimum : navigation complète au clavier et compatibilité avec un lecteur d'écran, dans l'esprit des recommandations RGAA/WCAG.

V. Recommandations en termes de sécurité

- **Authentification :** connexion sans mot de passe par lien magique, unique et temporaire, envoyé par e-mail ; session maintenue par un token JWT avec expiration. Aucun mot de passe n'est stocké.
- **Protection des routes :** toutes les routes privées sont protégées par un middleware JWT ; chaque restaurateur n'accède qu'à ses propres données.
- **Changement d'adresse e-mail :** double confirmation, avec un lien envoyé à la nouvelle adresse et une validation demandée à l'ancienne.
- **Services externes :** clés d'accès (Deliveroo, Instagram) stockées dans des variables d'environnement (fichier .env, exclu du dépôt Git) et utilisées uniquement côté back-end ; autorisation Instagram via OAuth 2.0, limitée à la publication.
- **Fichiers envoyés :** Multer n'accepte que des images, avec une taille maximale de 2 Mo.
- **Liens externes :** ouverture dans un nouvel onglet avec l'attribut noopener.
- **Dépendances :** audit régulier avec npm audit.
- **Communication :** HTTPS obligatoire sur l'ensemble du site.

VI. Maintenance du site et futures mises à jour

- **Mises à jour :** mises à jour régulières des dépendances front-end et back-end, avec application prioritaire des correctifs de sécurité.
- **Sauvegardes :** sauvegardes régulières de la base de données MongoDB.
- **Contrat de maintenance :** grandes lignes à définir avec Qwenta (fréquence des interventions et délai de réponse en cas d'incident).
- **Évolutions envisagées** (à planifier avec Qwenta) : intégration des tarifs directement dans MenuMaker, changement de moyen de paiement, blog interne à MenuMaker, animations sur la page d'accueil.
- **Points à confirmer avec Qwenta :** URL de la page tarifs et du back-office d'impression, existence d'un flux RSS pour le blog, accès partenaire à l'API Deliveroo.